

TP Neo4j

I. Utilisation de la console en ligne de Neo4j

2. Créer le graphe composé d'un nœud représentant un acteur (Type du nœud : Actor) nommé Tom Hanks (attribut name), d'un film (Movie) nommée Sleepless In Seattle et une arête entre l'acteur et le film de type "A joué dans" (Acted_in), de l'acteur vers le film

```
Query:
CREATE (Tom:Actor { name:'Tom Hanks' }), (Seattle:Movie { title:'Sleepless IN Seattle' }), (Tom)-[:ACTED_IN]->(Seattle)

Query took 7 ms and returned no rows.
Updated the graph - created 2 nodes and 1 relationship set 2 properties Result Details
```



3. Ecrire et exécuter la requête permettant de retrouver cet acteur (en affichant juste son nom)

```
Query:
MATCH (actor:Actor) WHERE actor.name = 'Tom Hanks' return actor
```

actor
(6:Actor {name:"Tom Hanks"})

Query took 7 ms and returned 1 rows. [Result Details](#)

4. Modifier la requête précédente afin de visualiser l'arête avec le film associé.

```
Query:
MATCH (actor:Actor)-[a:ACTED_IN*]-(m) WHERE actor.name = 'Tom Hanks' return actor,a,m
```

actor	a	m
(6:Actor {name:"Tom Hanks"})	[(6)-[6:ACTED_IN]->(7)]	(7:Movie {title:"Sleepless IN Seattle"})

Query took 12 ms and returned 1 rows. [Result Details](#)

5. Ecrire et exécuter la requête permettant de retrouver l'acteur Tom Hanks et e lui ajouter un attribut DoB de valeur 1944 à l'acteur Tom Hanks

```
Query:
MATCH (actor:Actor) WHERE actor.name = 'Tom Hanks' SET actor.DoB = 1944 RETURN actor.name, actor.DoB
```

actor.name	actor.DoB
Tom Hanks	1944

Query took 11 ms and returned 1 rows.
Updated the graph - set 1 property [Result Details](#)

6. Créer un nouveau Film de nom 'Forrest Gump'.

```
Query:
CREATE (Forrest:Movie { title:'Forrest Gump' })
```

Query took 4 ms and returned no rows.

Updated the graph - created 1 node set 1 property [Result Details](#)

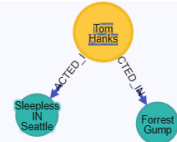


7. Associer le nœud Tom Hanks à ce nouveau film.

```
Query:
MATCH (Tom:Actor {name: 'Tom Hanks'}), (Forrest:Movie { title:'Forrest Gump' }) CREATE (Tom)-[:ACTED_IN]->(Forrest)
```

Query took 10 ms and returned no rows.

Updated the graph - created 1 relationship [Result Details](#)

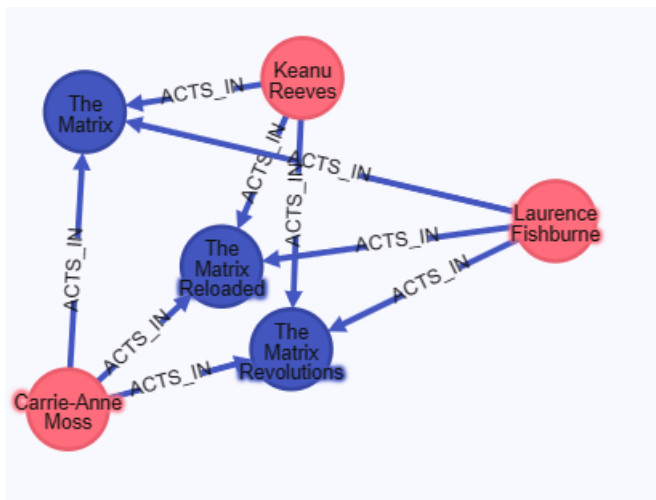


8. Réinitialiser la base et créer un nouveau graphe

```
Query:
CREATE (matrix1:Movie { title : 'The Matrix', year : '1999-03-31' }) CREATE (matrix2:Movie { title : 'The Matrix Reloaded', year : '2003-05-07' }) CREATE (matrix3:Movie { title : 'The Matrix Revolutions', year : '2003-10-27' }) CREATE (keanu:Actor { name:'Keanu Reeves' }) CREATE (carrieanne:Actor { name:'Carrie-Anne Moss' }) CREATE (keanu)-[:ACTS_IN { role : 'Neo' }]->(matrix1) CREATE (keanu)-[:ACTS_IN { role : 'Neo' }]->(matrix2) CREATE (keanu)-[:ACTS_IN { role : 'Neo' }]->(matrix3) CREATE (laurence)-[:ACTS_IN { role : 'Morpheus' }]->(matrix1) CREATE (laurence)-[:ACTS_IN { role : 'Morpheus' }]->(matrix2) CREATE (laurence)-[:ACTS_IN { role : 'Morpheus' }]->(matrix3) CREATE (carrieanne)-[:ACTS_IN { role : 'Trinity' }]->(matrix1) CREATE (carrieanne)-[:ACTS_IN { role : 'Trinity' }]->(matrix2) CREATE (carrieanne)-[:ACTS_IN { role : 'Trinity' }]->(matrix3)
```

Query took 15 ms and returned no rows.

Updated the graph - created 6 nodes and 9 relationships set 18 properties [Result Details](#)



9. Ecrire et exécuter la requête permettant de récupérer les films dont le titre contient 'Matrix'

```
Query:
MATCH (m:Movie) WHERE m.title CONTAINS "Matrix" RETURN m
```

m
(1:Movie {title:"The Matrix", year:"1999-03-31"})
(2:Movie {title:"The Matrix Reloaded", year:"2003-05-07"})
(3:Movie {title:"The Matrix Revolutions", year:"2003-10-27"})

Query took 16 ms and returned 3 rows.

[Result Details](#)

10. Ecrire et exécuter la requête permettant de récupérer les films dont le titre contient 'Matrix Re'

Query:

```
MATCH (m:Movie) WHERE m.title =~ '.*Matrix R.*' RETURN m
```

m
(41:Movie {title:"The Matrix Reloaded", year:"2003-05-07"})
(42:Movie {title:"The Matrix Revolutions", year:"2003-10-27"})

Query took 6 ms and returned 2 rows. [Result Details](#)

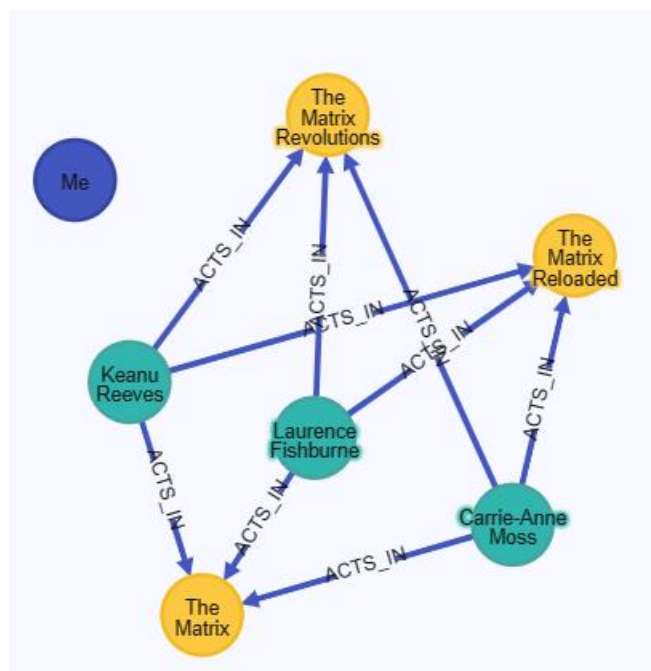
11. Créer un l'utilisateur (User), avec le champ nom (name) 'Me'

Query:

```
CREATE (n:User { name:'Me' })
```

Query took 5 ms and returned no rows.

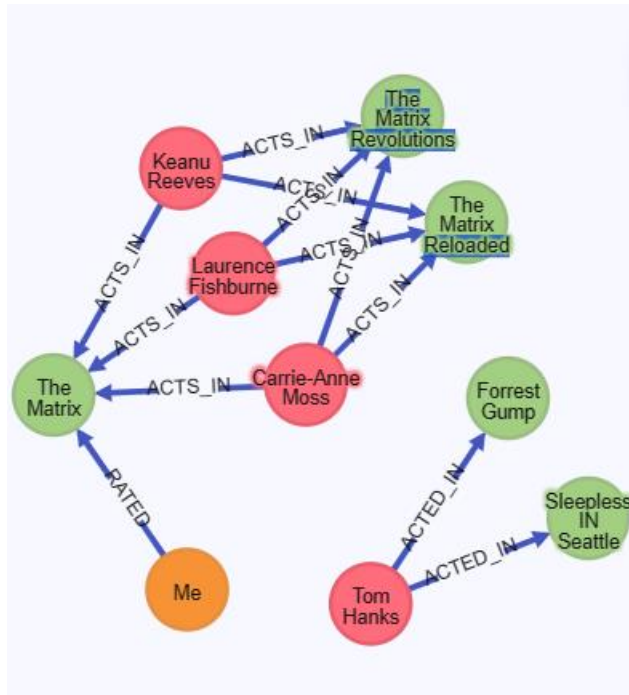
Updated the graph - created 1 node set 1 property [Result Details](#)



12. Ecrire et exécuter la requête permettant de récupérer l'utilisateur créé précédemment (à partir de son nom), un film à partir de son titre (que vous choisirez) et d'ajouter une arête entre le film et l'utilisateur, nommé RATED, et ayant pour attribut stars de valeur 5 et comment de valeur 'I love that movie!'.

Query:
 MATCH (me:User),(movie:Movie) WHERE me.name = 'Me' AND movie.title = 'The Matrix' CREATE (me)-[:RATED { stars : 5, comment : 'I love that movie!' }]->(movie)

Query took 11 ms and returned no rows.
 Updated the graph - created 1 relationship set 2 properties [Result Details](#)



13. Ecrire et exécuter la requête permettant de récupérer l'ensemble des titres de films notés par l'utilisateur, avec le nombre d'étoiles et le commentaire associé.

Query:
 MATCH (me:User),(me)-[rating:RATED]->(movie) WHERE me.name = 'Me' RETURN movie.title, rating.stars, rating.comment;

movie.title	rating.stars	rating.comment
The Matrix	5	I love that movie!

Query took 10 ms and returned 1 rows. [Result Details](#)

14. Ecrire et exécuter la requête permettant de récupérer l'ensemble des titres de films qui ne sont pas notés par l'utilisateur Me

```
Query:
MATCH (movie:Movie), (me:User {name: 'Me'}) WHERE NOT (me)-->(movie) RETURN movie.title
```

movie.title
Sleepless IN Seattle
Forrest Gump
The Matrix Reloaded
The Matrix Revolutions

Query took 15 ms and returned 4 rows. [Result Details](#)

15. Ecrire et exécuter la requête permettant de récupérer l'ensemble des titres de films qui sont notés par l'utilisateur Me et la liste des acteurs du film

```
Query:
MATCH (me:User {name: 'Me'}), (me)-[:RATED]-(movie), (actor)-[:ACTS_IN]-(movie) RETURN movie.title, collect(actor) AS actors
```

movie.title	actors
The Matrix	[(45:Actor {name:"Carrie-Anne Moss"}), (43:Actor {name:"Keanu Reeves"}), (44:Actor {name:"Laurence Fishburne"})]

Query took 13 ms and returned 1 rows. [Result Details](#)

16. Ecrire et exécuter la requête permettant de calculer le nombre de films par acteur (et ajouter un attribut moviesCount aux acteurs). Vérifier la mise à jour en affichant tous les acteurs

```
Query:
MATCH (a:Actor)-[:ACTS_IN]-(m) WITH a, count(m) AS moviesCount SET a.moviesCount = moviesCount RETURN a.moviesCount
```

a.moviesCount
3
3
3

Query took 14 ms and returned 3 rows.

Updated the graph - set 3 properties [Result Details](#)

L'attribut a bien été ajouté aux nœuds Actor :

```
Query:
MATCH (a:Actor) RETURN a
```

a
(0:Actor {DoB:1944, name:"Tom Hanks"})
(43:Actor {moviesCount:3, name:"Keanu Reeves"})
(44:Actor {moviesCount:3, name:"Laurence Fishburne"})
(45:Actor {moviesCount:3, name:"Carrie-Anne Moss"})

Query took 4 ms and returned 4 rows. [Result Details](#)

17. Ecrire et exécuter la requête permettant de calculer le nombre de commentaires par film (et ajouter un attribut commentsCount aux films). Vérifier la mise à jour en affichant tous les films.

```
Query:
MATCH (m:Movie)-[:RATED]-(u:User) WITH m, count(u) AS commentsCount SET m.commentsCount=commentsCount RETURN m.title, m.commentsCount
```

m.title	m.commentsCount
The Matrix	1

Query took 11 ms and returned 1 rows.
Updated the graph - set 1 property [Result Details](#)

L'attribut a bien été ajouté au nœud Movie :

```
Query:
MATCH (m:Movie) RETURN m
```

m
{1:Movie {title:"Sleepless IN Seattle"}}
{20:Movie {title:"Forrest Gump"}}
{40:Movie {commentsCount:1, title:"The Matrix", year:"1999-03-31"}}
{41:Movie {title:"The Matrix Reloaded", year:"2003-05-07"}}
{42:Movie {title:"The Matrix Revolutions", year:"2003-10-27"}}

Query took 6 ms and returned 5 rows. [Result Details](#)

18. Exécuter la requête suivante permettant de récupérer l'ensemble du graphe (nom, source et destination) :

```
Query:
START n=node(*) MATCH (n)-[r]->(m) RETURN n AS FROM , type(r) AS `->`, m AS to
```

Show entries Search:

FROM	->	to
{0:Actor {DoB:1944, name:"Tom Hanks"}}	ACTED_IN	{20:Movie {title:"Forrest Gump"}}
{0:Actor {DoB:1944, name:"Tom Hanks"}}	ACTED_IN	{1:Movie {title:"Sleepless IN Seattle"}}
{43:Actor {moviesCount:3, name:"Keanu Reeves"}}	ACTS_IN	{42:Movie {title:"The Matrix Revolutions", year:"2003-10-27"}}
{43:Actor {moviesCount:3, name:"Keanu Reeves"}}	ACTS_IN	{41:Movie {title:"The Matrix Reloaded", year:"2003-05-07"}}
{43:Actor {moviesCount:3, name:"Keanu Reeves"}}	ACTS_IN	{40:Movie {commentsCount:1, title:"The Matrix", year:"1999-03-31"}}
{44:Actor {moviesCount:3, name:"Laurence Fishburne"}}	ACTS_IN	{40:Movie {commentsCount:1, title:"The Matrix", year:"1999-03-31"}}
{44:Actor {moviesCount:3, name:"Laurence Fishburne"}}	ACTS_IN	{42:Movie {title:"The Matrix Revolutions", year:"2003-10-27"}}
{44:Actor {moviesCount:3, name:"Laurence Fishburne"}}	ACTS_IN	{41:Movie {title:"The Matrix Reloaded", year:"2003-05-07"}}
{45:Actor {moviesCount:3, name:"Carrie-Anne Moss"}}	ACTS_IN	{41:Movie {title:"The Matrix Reloaded", year:"2003-05-07"}}
{45:Actor {moviesCount:3, name:"Carrie-Anne Moss"}}	ACTS_IN	{40:Movie {commentsCount:1, title:"The Matrix", year:"1999-03-31"}}

Showing 1 to 10 of 12 entries >>
Query took 15 ms and returned 12 rows. [Result Details](#)

19. Exécuter la requête suivante permettant de compter les nœuds du graphe via l'instruction :

```
Query:
start n=node(*) return count(n);
```

count (n)
10

Query took 10 ms and returned 1 rows. [Result Details](#)

20. Dans un navigateur web, visualiser le contenu du fichier 2 :

<https://gist.githubusercontent.com/jexp/d788e117129c3730a042/raw/c5f98f269b4a219ebb88bd3eb83d331f439a8201/people.csv>

